Applied Soft Computing Journal 186 (2026) 114223

Contents lists available at ScienceDirect

Applied Soft Computing

journal homepage: www.elsevier.com/locate/asoc

Graph variational autoencoder with affinity propagation for community-aware anomaly detection in attributed networks

Zhijie Cao[a], Chengkun Yang[a], Xiaoqing Fan[a], Lingjie Li[b], Qiuzhen Lin[a], Jianqiang Li[c], Lijia Ma[a,*]

[a] College of Computer Science and Software Engineering, Shenzhen University, Shenzhen, 518060, China
[b] Guangdong Laboratory of Artificial Intelligence and Digital Economy, Shenzhen, 518060, China
[c] National Engineering Laboratory for Big Data System Computing Technology, Shenzhen University, Shenzhen, 518060, China

HIGHLIGHTS

- Proposed a method to learn low-dimensional node representations using link and attribute information.
- Developed a mechanism to detect communities based on structural and attribute similarities.
- Designed an anomaly score calculation integrating link structure, node attributes, and communities.
- The proposed model outperforms other methods in anomaly detection performance.

ARTICLE INFO

Keywords:
Anomaly detection
Attributed network
Community detection
Graph variational autoencoder
Affinity propagation

ABSTRACT

Anomaly detection in attributed networks (ADAN) aims to identify abnormal nodes that exhibit unexpected link structures and attributes compared to the others. The existing works primarily utilize the node representations learned from the low-level attributes and link structures of nodes to detect abnormal nodes, which overlook the impact of high-level community structures on ADAN. To address this issue, this paper proposes a novel framework called Graph Variational Autoencoder with Affinity Propagation (GVE-AP) for community-aware ADAN. GVE-AP first employs a graph convolutional variational autoencoder to learn node embeddings from attributed networks. Then, it integrates an affinity propagation algorithm for community division, which jointly considers both node attributes and link structures. Subsequently, it introduces a novel community-aware anomaly score to detect abnormal nodes by measuring dissimilarity with their communities based on robust features extracted via principal component analysis. Experimental results on eight real-world datasets demonstrate that GVE-AP outperforms the state-of-the-art methods for anomaly detection in attributed networks in terms of AUC and robustness.

1. Introduction

An attributed network is composed of sets of nodes, links and node attributes, and it is widely used for representing real-world systems in various domains such as social sciences [1], biology [2], and finance [3]. Anomaly detection in attributed networks (ADAN) aims to identify abnormal nodes that exhibit significant deviations or abnormalities in the link structures or attributes compared with the others [4,5]. The studies on ADAN are essential for detecting potential anomalies, fraud, and abnormal behaviors, thereby enhancing security and reliability of systems, which have been widely applied in various domains, including financial fraud detection [6], rumor identification [7], and network intrusion detection [8].

Existing ADAN methods are broadly divided into two categories: traditional shallow-learning methods and deep-learning-based methods. Traditional shallow-learning methods, such as LOF [9], CODA [10], Radar [11], and OddBall [12], rely on statistical, density-based, or distance-based metrics. Although these methods are computationally simple and efficient, they often struggle to capture complex, non-linear interactions within high-dimensional or large-scale networks.

* Corresponding author.
Email addresses: caozj2003@gmail.com (Z. Cao), yangchengkun234@gmail.com (C. Yang), 2213875895@qq.com (X. Fan), lilingjie2017@email.szu.edu.cn (L. Li), qiuzhlin@szu.edu.cn (Q. Lin), lijq@szu.edu.cn (J. Li), omegamalj@gmail.com (L. Ma).

https://doi.org/10.1016/j.asoc.2025.114223
Received 7 November 2024; Received in revised form 19 September 2025; Accepted 1 November 2025
Available online 12 November 2025
1568-4946/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

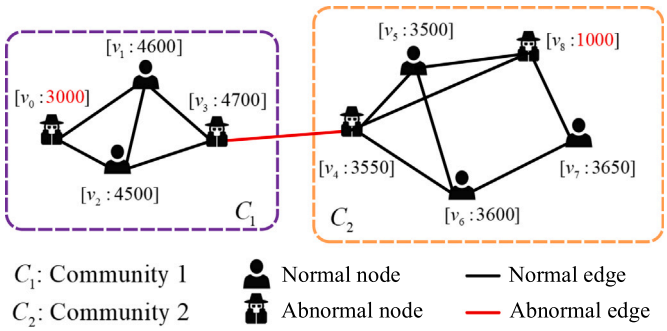


Fig. 1. A toy example of community-aware anomaly node detection in attributed networks.

In recent years, deep learning models have emerged to address the aforementioned issue. Representative methods such as DOMINANT [13], SpecAE [14], DGI [15], DeepAnT [16], Anomalydae [17], CoLA [18], ComGA [19], ALARM [20], CAFD [21], CD-GAD [22], and REL-BC [23] mainly utilize graph autoencoders to uncover key node features with structure and attribute information in the low-dimensional representation, and detect these nodes with obviously different features as anomalous ones. By modeling node interactions, these methods can capture complex dependencies and contextual information, making them suitable for tackling large-scale heterogeneous networks [13,24,25]. However, most of them focus on global modeling and ignore the effects of community structures in attributed networks, failing to fully capture the distinctive behavioral patterns [5,26].

Community structures are latent functional groups within networks [27–32]. Dividing the network into communities enables clustering of similar nodes and reveals potential substructures [33–36]. Community-aware anomaly detection aims to identify nodes that exhibit significant differences in structure or attributes compared with other nodes in the same community. A toy example of community-aware anomaly detection is shown in Fig. 1, where nodes v_0 and v_8 are identified as attribute anomalies, while nodes v_3 and v_4 are structural anomalies due to cross-community connections. In recent years, several methods have been proposed for community detection in attributed networks [37–40], but many of these approaches may rely on a predefined number of communities. To address this issue, methods such as DAG [41] and Sobolevsky and Belyi's GNN-based algorithm [42] have been proposed. DAG uses deep learning to adaptively detect communities and estimate their number, while [42] performs unsupervised community detection via modularity optimization on learned node embeddings.

To address these challenges and inspired by advances in community detection, we propose a Graph Variational Autoencoder with Affinity Propagation (GVE-AP) framework. GVE-AP employs a Graph Variational Autoencoders (GVE) to extract features from the network data, simultaneously incorporating both structure and attribute information. Compared to traditional VGAE method [43], GVE enhances feature extraction by integrating Principal Component Analysis (PCA) [44] and a self-attention mechanism, thereby improving the representations of structure and attribute information. Based on the learned node features, the Affinity Propagation (AP) clustering algorithm [45,46] is utilized for community detection without requiring a predefined number of communities. Subsequently, PCA is applied to reduce the dimensionality of both structural and attribute features, and a novel anomaly score function is designed based on these reduced features. Finally, anomaly scores for each node are calculated using the community detection results. Extensive experiments on eight real-world attributed network datasets demonstrate the effectiveness and robustness of the proposed framework. The main contributions of GVE-AP for the ADAN task are summarized as follows:

- We propose a novel framework called GVE-AP, which combines Graph Variational Autoencoders (GVE) with Affinity Propagation (AP) clustering, and designs a new community-aware anomaly score function to detect anomalous nodes from a community perspective.
- We enhance a GVE model by integrating PCA and a self-attention mechanism to learn low-dimensional and expressive node representations that effectively capture both structure and attribute information.
- Extensive experiments on eight real-world attributed network datasets show that the GVE-AP outperforms eight state-of-the-art methods in terms of AUC, validating its robustness and effectiveness for the ADAN task.

2. Related work

The concept of anomalies or outliers was first introduced by Grubbs [4], aiming to identify observations that significantly deviate from the majority of ones. To address this problem, the anomaly detection (AD) techniques have been widely applied for risk identification in various systems, garnering considerable attention. Traditional AD algorithms, based on statistics, density, or distance metrics, primarily detect anomalies through anomaly scoring models. For instance, LOF evaluates the anomaly degree of a data point by computing the local density ratio between the point and its neighbors [9]. CODA measures cluster compactness and separability to identify outliers [10]. Radar detects anomalies by comparing the residuals between actual and predicted attribute values [11]. AMEN ranks neighbor anomalies using attribute similarity [47]. ANOMALOUS jointly models attribute similarity and structural aggregation to identify anomalous nodes [48]. OddBall defines various models for characterizing node anomalies within Ego-networks [12]. However, these traditional methods encounter limitations when addressing complex, high-dimensional network data.

Recently, deep learning-based methods have gained popularity in AD due to their ability to learn low-dimensional node representations effectively. For example, DOMINANT utilizes graph autoencoders to calculate anomaly scores based on reconstruction errors of attributes and structures [13]. CoLA applies contrastive self-supervised learning to amplify differences between normal and abnormal nodes [18]. GRADATE leverages the original graph and an edge-augmented graph as two views, performing multi-scale contrastive learning (node-subgraph, node-node, and subgraph-subgraph) to detect anomalous nodes [49]. PREM performs anonymized message passing once in preprocessing, uses an ego-neighbor matching module to compute cosine similarity between mapped embeddings, and takes the negative similarity as the node anomaly score, enabling efficient detection with low time and memory cost [50]. GADMCLG adopts a multiscale contrastive self-supervised framework that integrates local and global contrasts, introduces subgraph centralization to reduce positional bias, and detects node anomalies through weighted aggregation of anomaly scores [51]. Moreover, methods like AnomalyLLM [52] and LogGPT [53] combine large language models with contextual information for AD. Specifically, AnomalyLLM achieves few-shot anomaly detection through dynamic-aware pre-training and in-context learning, while LogGPT enhances AD performance via sequential modeling.

Furthermore, community detection in attributed networks has been extensively studied by taking node attributes and network topology into consideration. For example, NEC learns node representations preserving both structure and attributes for effective community detection [38]. ComGA embeds community modularity features into graph autoencoders using Infomap [19]. SpecF incorporates community information into the Laplacian matrix to enhance AD performance [40]. While SpecF enhances anomaly detection through spectral filtering with community structure, it is sensitive to filter parameters and community settings and lacks unified fusion of structural and attribute anomalies. ComGA models multiple anomaly types but introduces complexity and relies heavily on accurate community partitions. In contrast, our GVE-AP employs parameter-free AP clustering, community-aware anomaly scoring, PCA-based denoising, and balanced fusion of attribute and structural

anomalies, achieving greater robustness, accuracy, and interpretability. In addition, other community detection approaches have also been explored, such as stochastic block model-based methods and matrix factorization-based methods [54].

3. GVE-AP: the proposed approach

3.1. Problem definition

We let $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{X})$ denote an attributed network $\mathcal{G}$ with a node set $\mathcal{V}$, an edge set $\mathcal{E}$, and an attribute matrix $\mathbf{X}$. The link structures $\mathcal{E}$ of nodes in $\mathcal{V}$ are represented by an adjacency matrix $\mathbf{A} = [a_{ij}]_{n\times n}$, where $a_{ij} \in \{0, 1\}$. Specifically, $a_{ij} = 1$ ($a_{ij} = 0$) indicates the presence (absence) of a link between nodes v_i and v_j, while n is the total number of nodes. Additionally, the node attributes in $\mathcal{V}$ are represented by the attribute matrix $\mathbf{X} = [\mathbf{x}_i]_{n\times d}$, where i-th row vector $\mathbf{x}_i$ denotes the attribute information of node v_i, while d is the dimensionality of attribute information. Table 1 summarizes the notations used in the paper.

Anomalous Nodes and Edges: In community-aware anomaly detection, anomalous nodes denote nodes whose structural or attribute patterns significantly deviate from their community. Abnormal edges represent edges that deviate from expected intra-community connections, including cross-community edges or edges with unusual structural or attribute patterns.

Anomaly Detection in Attributed Networks (ADAN): Given an attributed network $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{X})$, ADAN aims to learn an anomaly score function $\mathcal{F}$ that accurately calculates the anomaly score $y_i = \mathcal{F}(v_i)$ for each node v_i. In this case, anomalous nodes can be identified by ranking all nodes according to their anomaly scores.

Community-aware ADAN: This task aims to learn a community-aware anomaly function $\mathcal{F}$ that enables to calculation of the anomaly score y for each node compared with its communities in link structures and attributes.

3.2. GVE-AP framework

The proposed GVE-AP for ADAN consists of GVE, AP, and an anomaly score design technique. GVE is used to learn the low-dimensional node representations of attributed networks, taking link structures and node attributes into consideration. Meanwhile, AP is utilized to identify communities within the attributed networks based on these learned node representations. The anomaly score design technique is proposed to evaluate the anomaly scores of nodes, considering link structures, attributes and communities of nodes. Fig. 2 depicts the schematic diagram of the GVE-AP algorithm.

Table 1
Notation summary.

Notation	Explanation
$\mathcal{G}$	An attributed network
$\mathcal{V}$	The node set of $\mathcal{G}$
$\mathcal{E}$	The edge set of $\mathcal{G}$
$\mathbf{A}$	The adjacency matrix of $\mathcal{G}$
$\mathbf{X}$	The attribute matrix of $\mathcal{G}$
$\mathbf{I}$	The identity matrix
$\mathbf{x}_i$	The attribute information of node v_i
$\widetilde{\mathbf{D}}$	The diagonal degree matrix
$\mathbf{Z}$	The attributed network embeddings
$\mathbf{H}^{(l)}$	The l-th layer hidden representation matrix
$\mathbf{W}^{(l)}$	The l-th layer weight matrix
$\mathbf{W}_q$	The query weight matrix
$\mathbf{W}_k$	The key weight matrix
$\mathbf{W}_v$	The value weight matrix
$\mathcal{L}$	The loss function
$\mathbf{D}$	The similarity matrix
$\mathbf{R}$	The responsibility matrix
$\mathbf{M}$	The availability matrix
$\mathbf{S}$	The score matrix
$\mathcal{F}(\cdot)$	The anomaly score function
$\mathcal{F}_a^i$	The structure anomaly score of node v_i
$\mathcal{F}_x^i$	The attribute anomaly score of node v_i
C_i	The community affiliation of node v_i
n	The number of nodes
d	The dimensionality of attribute information
α	The hyperparameter

3.3. Graph variational autoencoder

3.3.1. Graph convolutional encoder

The graph convolutional encoder propagates and aggregates information within local neighborhoods. It considers both node features and the features of their neighbors. By stacking multiple graph convolutional layers, it extracts higher-level structural and attribute features. The encoder embeds nodes into a low-dimensional space, preserving key information while removing noise.

For the network structure, we normalize the original adjacency matrix $\mathbf{A}$ to reduce the influence of degree differences and obtain more stable and discriminative node embeddings. The normalization process is formulated as follows:

$$\widetilde{\mathbf{A}} = \widetilde{\mathbf{D}}^{-\frac{1}{2}}(\mathbf{A} + \mathbf{I})\widetilde{\mathbf{D}}^{-\frac{1}{2}}, \tag{1}$$

where $\mathbf{A}$ is the original adjacency matrix, $\mathbf{I}$ denotes the identity matrix, $\widetilde{\mathbf{D}}$ is a diagonal matrix with diagonal elements $\widetilde{\mathbf{D}}_{ii} = \sum_j(\mathbf{A}_{ij} + \mathbf{I}_{ij})$, and $\widetilde{\mathbf{A}}$ represents the normalized adjacency matrix.

For the network attributes, we apply the PCA [44] to reduce the dimensionality of the input attribute information, thereby obtaining attribute features with unified dimensions. PCA expresses the attribute information using fewer principal components, which helps to remove noise and improve computational efficiency. The process is defined as follows:

$$\mathbf{X}' \leftarrow \mathbf{PCA}(\mathbf{X}), \tag{2}$$

where $\mathbf{PCA}(\cdot)$ denotes the PCA method, $\mathbf{X}$ is the original attribute information, and $\mathbf{X}'$ represents the dimension-reduced attribute features. Then, $\mathbf{X}'$ is used as the initial node feature representations for the encoder:

$$\mathbf{H}^{(0)} = \mathbf{X}'. \tag{3}$$

Subsequently, the graph convolutional encoder computes the feature representation of each node in the next layer through the propagation rule. After m iterations of updates, the final node feature representations are obtained. This process can be expressed as:

$$\mathbf{H}^{(l+1)} = \mathbf{ReLU}(\widetilde{\mathbf{A}}\mathbf{H}^{(l)}\mathbf{W}^{(l)}), \tag{4}$$

$$\mathbf{Z} = \mathbf{H}^{(m)}, \tag{5}$$

where $\mathbf{H}^{(l)}$ denotes the node feature matrix of the l-th layer, $\mathbf{W}^{(l)}$ represents the weight matrix of the l-th layer, m-th layer is the last layer, and $\mathbf{Z}$ is the final node feature matrix. In our implementation, the encoder comprises two graph convolutional layers ($m = 2$). $\mathbf{ReLU}(\cdot)$ denotes the activation function, which sets negative feature values to zero to capture nonlinear relationships in the data.

3.3.2. Graph convolutional decoder

The graph convolutional decoder reconstructs the data from the low-dimensional latent space back to the high-dimensional space, enabling the extraction of key features and learning of compact and informative node representations. It consists of two components: a one-layer structure decoder and a two-layer attribute decoder.

For the structure decoder, a self-attention mechanism is employed to capture dependencies between different types of nodes and reconstruct the network structure for subsequent community detection. Specifically, the structural decoder maps the encoded latent representations $\mathbf{Z}$ into

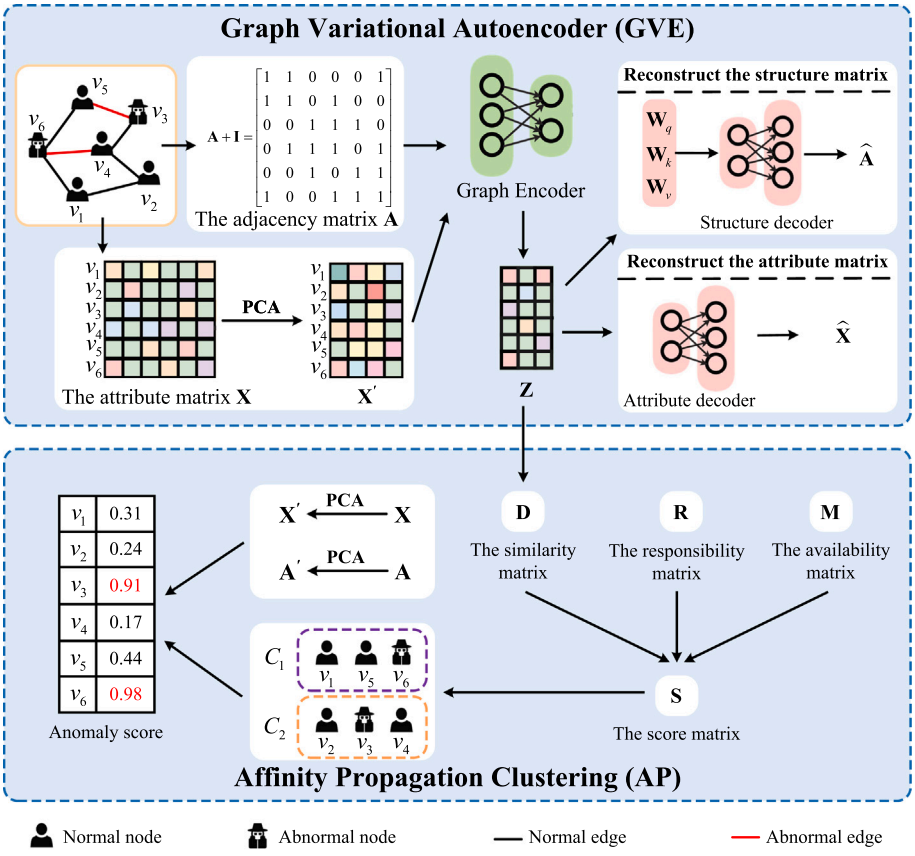


Fig. 2. Overview of the GVE-AP framework.

a new space via the self-attention mechanism. Then, the resulting attention matrix is normalized using the **Softmax**(·) function for node-link prediction, formulated as:

$$\widetilde{\mathbf{Z}} = \text{Softmax}\left(\frac{\mathbf{Z}\mathbf{W}_q(\mathbf{Z}\mathbf{W}_k)^T}{\sqrt{d_k}}\right)(\mathbf{Z}\mathbf{W}_v), \tag{6}$$

where $\mathbf{W}_q$, $\mathbf{W}_k$, and $\mathbf{W}_v$ are the query, key, and value weight matrices, respectively. d_k represents the feature scaling dimension, which controls the range of attention weights to ensure stability and effectiveness. $\widetilde{\mathbf{Z}}$ denotes the reconstructed node embeddings. Subsequently, $\widetilde{\mathbf{Z}}$ is fed into the **Sigmoid**(·) function to compute the probability of a link existing between each pair of nodes, which is formulated as:

$$P(\hat{\mathbf{A}}_{ij} = 1 \mid (\tilde{\mathbf{z}}_i, \tilde{\mathbf{z}}_j)) = \textbf{Sigmoid}(\tilde{\mathbf{z}}_i, \tilde{\mathbf{z}}_j^{\mathrm{T}}), \tag{7}$$

where $\hat{\mathbf{A}}$ denotes the reconstructed adjacency matrix, and $\tilde{\mathbf{z}}_i$ denotes the reconstructed embedding vector of node i. The **Sigmoid**(·) function maps the reconstructed embeddings to the range $[0, 1]$, generating the probability of a link existing between nodes.

For the attribute decoder, given the encoder output $\mathbf{Z}$, the node attributes are reconstructed through two graph convolutional layers as follows:

$$\hat{\mathbf{X}} = \textbf{ReLU}(\mathbf{A}\mathbf{Z}\mathbf{W}^{(l')}), \tag{8}$$

where $\mathbf{W}^{(l')}$ denotes the weight matrix of the l'-th layer in the attribute decoder. Since the original node attributes are strictly non-negative, the **ReLU**(·) function is applied to eliminate negative values during reconstruction, thereby preserving consistency with the non-negativity of the original attributes.

3.3.3. The loss function

The loss function of GVE consists of three components: the structural reconstruction loss, the attribute reconstruction loss, and the KL divergence term, defined as follows:

$$\mathcal{L} = (1-\alpha)||\mathbf{A} - \hat{\mathbf{A}}||_F^2 + \alpha||\mathbf{X}' - \hat{\mathbf{X}}||_F^2 + \textbf{KL}(q(\mathbf{Z} \mid (\mathbf{A}, \mathbf{X}')) \mid\mid p(\mathbf{Z})), \tag{9}$$

where $||\mathbf{A} - \hat{\mathbf{A}}||_F^2$ denotes the structural reconstruction loss, and $||\mathbf{X}' - \hat{\mathbf{X}}||_F^2$ represents the attribute reconstruction loss. Their trade-off is controlled by the hyperparameter α. The **KL** divergence term serves as the core variational component, regularizing the latent distribution $q(\mathbf{Z} \mid (\mathbf{A}, \mathbf{X}'))$ to approximate the prior distribution $p(\mathbf{Z}) = \mathcal{N}(0, \mathbf{I})$. Here, $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ denote the mean vector and standard deviation vector of the latent representation $\mathbf{Z}$, which are learned by the encoder from the input adjacency matrix $\mathbf{A}$ and attribute features $\mathbf{X}'$. This incorporation of variational inference enhances the generalization ability of the model and effectively prevents overfitting.

3.4. Affinity propagation clustering

To effectively leverage community discovery for anomaly detection in attributed networks, GVE-AP uses the AP clustering algorithm [45] for community partitioning. The algorithm does not require a predefined number of communities and assigns nodes to community centers selected via message passing based on their similarity. Unlike the original AP clustering algorithm, GVE-AP constructs the similarity matrix using the network embeddings $\mathbf{Z}$ and adapts the objective from general clustering to community discovery for anomaly detection.

Specifically, after obtaining the network embeddings $\mathbf{Z}$ generated by GVE, the model uses them to compute the similarity matrix. $\mathbf{Z}$ integrates both structural and attribute information, which can capture

complex nonlinear relationships and high-order neighborhood dependencies, providing a more accurate measure of node-to-node similarity. The similarity matrix is defined as:

$$\mathbf{D}(i,j) = -||\mathbf{z}_i - \mathbf{z}_j||^2, \tag{10}$$

where $\mathbf{D}$ denotes the similarity matrix. Then, the AP clustering algorithm constructs a score matrix $\mathbf{S}$ to quantify the association between nodes and candidate community centers, thereby determining each node's optimal community assignment. The score matrix $\mathbf{S}$ is obtained by summing the responsibility matrix $\mathbf{R}$ and the availability matrix $\mathbf{M}$, which are initialized as follows:

$$\mathbf{R}(i,j) = 0,\ \mathbf{M}(i,j) = 0, \quad i,j \in \{0,\ 1,\ \cdots,\ n-1\}, \tag{11}$$

where $\mathbf{R}$ is the responsibility matrix, representing the extent to which node v_i chooses node v_j as a community center, and $\mathbf{M}$ is the availability matrix, representing the benefit gained by node v_i when selecting node v_j as a community center.

Subsequently, $\mathbf{R}$ and $\mathbf{M}$ are alternately updated until $\mathbf{S}$ converges. This process is formulated as:

$$\mathbf{R}(i,j) = \mathbf{D}(i,j) - \max_{k \in \mathcal{V} - v_j} \{\mathbf{D}(i,k) + \mathbf{M}(i,k)\}, \tag{12}$$

$$\mathbf{M}(i,j) = \min\left\{0, \mathbf{R}(j,j) + \max \sum_{k \in \mathcal{V} - v_i - v_j} \{0, \mathbf{R}(k,j)\}\right\}, \tag{13}$$

$$\mathbf{S}(i,j) = \mathbf{R}(i,j) + \mathbf{M}(i,j). \tag{14}$$

After the AP clustering algorithm converges, each node is assigned to its most suitable community, and these community labels will be used in the next subsection to compute the average Euclidean distance between a node and other nodes within the same community for anomaly detection.

3.5. Anomaly score design

In this subsection, we design a community-aware anomaly score function based on the Euclidean distance between a node and other nodes within the same community in terms of both attribute and structural information. A higher anomaly score indicates that the node is more anomalous with respect to its community.

GVE applies multiple nonlinear transformations during training, compressing node attributes and structural features into a low-dimensional space, which makes it difficult to disentangle the contributions of attribute and structural anomalies in the final anomaly score, thus reducing interpretability. In contrast, PCA extracts principal components based on statistical methods, offering a transparent and interpretable process. Furthermore, applying PCA to the adjacency matrix $\mathbf{A}$ alleviates the problems of high-dimensional sparsity and redundant noise, extracts global structural patterns, and enhances the stability of anomaly detection. Therefore, after community partitioning, we introduce PCA to reduce the dimensionality of both the attribute matrix $\mathbf{X}$ and the adjacency matrix $\mathbf{A}$. The dimension-reduced attribute matrix $\mathbf{X}'$ obtained from Eq. (2) is directly used, and the dimensionality reduction of the adjacency matrix is formulated as follows:

$$\mathbf{A}' \leftarrow \mathbf{PCA}(\mathbf{A}). \tag{15}$$

Then, based on the community partitioning obtained from the AP clustering algorithm, we compute the structural anomaly score and the attribute anomaly score for each node as follows:

$$\mathcal{F}_a^i = \frac{1}{|C_i|} \sum_{k \in C_i - v_i} ||\mathbf{a}_i' - \mathbf{a}_k'||^2, \tag{16}$$

$$\mathcal{F}_x^i = \frac{1}{|C_i|} \sum_{k \in C_i - v_i} ||\mathbf{x}_i' - \mathbf{x}_k'||^2, \tag{17}$$

where C_i denotes the community to which node v_i belongs, and $|C_i|$ represents the number of nodes in C_i. $\mathcal{F}_a^i$ and $\mathcal{F}_x^i$ denote the structural and attribute anomaly scores of node v_i, respectively.

Finally, $\mathcal{F}_a^i$ and $\mathcal{F}_x^i$ are normalized to ensure they are on comparable scales, preventing either score from dominating the final anomaly score. After normalization, the two scores are summed to obtain the final anomaly score for node v_i, defined as follows:

$$\mathcal{F}(v_i) = \frac{\mathcal{F}_x^i - \min \mathcal{F}_x}{\max \mathcal{F}_x - \min \mathcal{F}_x} + \frac{\mathcal{F}_a^i - \min \mathcal{F}_a}{\max \mathcal{F}_a - \min \mathcal{F}_a}, \tag{18}$$

where $\mathcal{F}(v_i)$ is the final anomaly score of node v_i. A node is identified as anomalous if its anomaly score is excessively large. Algorithm 2 and Fig. 3 describe the complete workflow of community-aware anomaly detection in detail.

3.6. Algorithm complexity analysis

In Algorithm 1, the most time-consuming operations are the dimensionality reduction of the feature vector matrix $\mathbf{X}$ using PCA and the iterative updates of the node representation $\mathbf{H}^{(l)}$ for g_1 iterations. The time complexities for these operations are $O(h \times n^2)$ and $O(g_1 \times n^2 \times d)$,

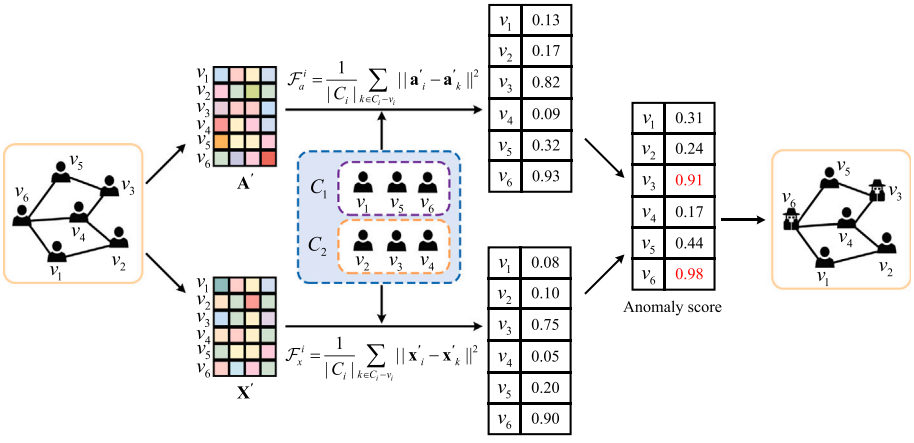


Fig. 3. Illustration of anomaly detection. $\mathbf{A}'$ and $\mathbf{X}'$ represent the structural matrix and attribute matrix, respectively, after undergoing PCA feature extraction. C_1 and C_2 are two distinct communities obtained through the AP algorithm partitioning. $\mathcal{F}_a$ represents the structure anomaly scores, while $\mathcal{F}_x$ represents the attribute anomaly scores.

Algorithm 1 Attributed network embeddings.

Input: An attributed network: $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{X})$, number of iterations: g_1, hyperparameter: α
Output: Attributed network embeddings: $\mathbf{Z}$

Calculate dimension reduction attribute features $\mathbf{X}'$ (see Eq. (2)).
Calculate the normalized adjacency matrix $\widetilde{\mathbf{A}}$ (see Eq. (1)).
for ($i = 1$ to g_1) **do**
  Calculate the hidden representation $\mathbf{H}^{(l)}$ for each layer (see Eq. (4)).
  Reconstruct the attribute matrix $\widehat{\mathbf{X}}$ (see Eq. (8)).
  Reconstruct the adjacency matrix $\widehat{\mathbf{A}}$ based on self-attention mechanism (see Eqs. (6) and (7)).
  Compute the loss function $\mathcal{L}$ and perform backpropagation (see Eq. (9)).
end for

Algorithm 2 Anomaly detection (AD).

Input: Attributed network embeddings: $\mathbf{Z}$, number of iterations: g_2
Output: Anomaly scores of all nodes: $\mathcal{F}$

Initialize the similarity matrix $\mathbf{D}$, responsibility matrix $\mathbf{R}$ and availability matrix $\mathbf{M}$ (see Eqs. (11) and (10)).
for ($i = 1$ to g_2) **do**
  Update responsibility matrix $\mathbf{R}$ (see Eq. (12)).
  Update availability matrix $\mathbf{M}$ (see Eq. (13)).
  Calculate the score of each node v_i relative to all cluster centers v_k (see Eq. (14)).
end for
Perform feature extraction on the adjacency matrix $\mathbf{A}$ using PCA (see Eq. (15)).
Calculate the outlier scores $\mathcal{F}_a^i$ for structure (see Eq. (16)).
Calculate the outlier scores $\mathcal{F}_x^i$ for attributes (see Eq. (17)).
Calculate the total outlier score $\mathcal{F}(v_i)$ (see Eq. (18)).

respectively, where h represents the number of input attribute features, n denotes the number of nodes, g_1 is the number of iterations in Algorithm 1, and d is the dimensionality of the node embeddings. In Algorithm 2, the main time consumption comes from updating the responsibility matrix $\mathbf{R}$ and the availability matrix $\mathbf{M}$, as well as calculating the score matrix $\mathbf{S}$. In each iteration, the time complexity for updating the responsibility matrix $\mathbf{R}$ and the availability matrix $\mathbf{M}$ is $O(n^2)$. Since Algorithm 2 undergoes g_2 iterations, its time complexity is $O(g_2 \times n^2)$. Thus, the overall time complexity of GVE-AP is $O(h \times n^2 + g_1 \times n^2 \times d + g_2 \times n^2)$.

4. Experiments

In this section, to validate the effectiveness of the proposed GVE-AP, we test its node AD performance on eight real-world attributed networks, and compare it with eight state-of-the-art baselines.

4.1. Experimental settings

4.1.1. Datasets

The experiments are conducted on eight real-world attributed networks, including four citation networks (ACM, Cora, Citeseer, and Pubmed), two social networks (BlogCatalog and Flickr) [13,18,55,56], one user-subreddit network (Reddit), and one work collaboration network (Tolokers). The citation networks record the authorship and citation relationships among academic papers, while the social networks reflect interpersonal connections, interests, and community structure. The Reddit dataset focuses on user-subreddit interactions, and the Tolokers dataset captures work collaboration activities. The statistical information of these networks is summarized in Table 2, and their details can be found in [13,18,56].

Table 2
Statistical information of datasets.

Dataset	Nodes	Edges	Attributes	Anomalies
ACM	16,484	71,980	8337	600
Cora	2708	5429	1433	150
Citeseer	3327	4732	3703	150
Pubmed	19,717	44,338	500	600
BlogCatalog	5196	171,743	8189	300
Flickr	7575	239,738	12,074	450
Reddit	10,984	168,016	64	366
Tolokers	11,758	519,000	10	2566

4.1.2. Baselines

Eight state-of-the-art methods are selected as baselines for comparisons with the GVE-AP, including four shallow algorithms (LOF [9], AMEN [47], Radar [11], and ANOMALOUS [48]) and four deep learning-based methods (DOMINANT [13], CoLA [18], ComGA [19], and GRADATE [49]).

- **LOF:** LOF evaluates the anomaly score of each point by comparing its local density to that of its neighbors.
- **AMEN:** AMEN ranks nodes by assessing attribute dissimilarities with their neighbors.
- **Radar:** Radar detects anomalies by comparing the residuals between nodes' observed and predicted attribute values.
- **ANOMALOUS:** ANOMALOUS jointly models attribute similarity and structural aggregation to identify anomalous nodes.
- **DOMINANT:** DOMINANT uses graph convolutional autoencoders to minimize both attribute and structure reconstruction errors for anomaly detection.
- **CoLA:** CoLA adopts graph contrastive learning to learn node representations by maximizing similarity among normal nodes and distinguishing anomalies.
- **ComGA:** ComGA embeds community modularity features into a graph autoencoder, and uses reconstruction errors for detecting anomalies.
- **GRADATE:** GRADATE proposes a multi-view multi-scale contrastive learning framework, which introduces a subgraph contrast to capture local structural differences between normal and abnormal subgraphs, enhancing node representation learning for anomaly detection.

4.1.3. Parameter settings

In all experiments, the graph convolutional encoder of GVE-AP consists of three layers with 64, 32, and 16 neurons, respectively. The number of principal components for PCA is set to 256. Moreover, the learning rate is set to 0.005, and the Adam optimizer is employed to optimize the loss function with 100 iterations, with the weighting factor α set to 0.8. In addition, for the AP clustering algorithm, the damping factor is set to 0.5, and the maximum number of iterations is set to 500. All baseline algorithms maintain the same parameter settings as described in their original papers.

4.2. Performance analysis

In this section, we evaluate the proposed GVE-AP algorithm along with eight benchmark algorithms for AD on eight real-world attributed networks. AUC is used as the performance metric. Furthermore, we conduct ablation studies to examine the contributions of structural anomalies, attribute anomalies, and community anomalies to the AD task. Finally, we analyze the impact of the hyperparameter α on the graph variational encoder loss function.

Table 3
The AUC values of AD algorithms on the eight datasets. The best performance on each dataset is highlighted in **bold**, while the second-best performance is underlined.

Methods	ACM	Cora	Citeseer	Pubmed	BlogCatalog	Flickr	Reddit	Tolokers	Average
LOF	0.4994	0.5030	0.4974	0.5003	0.5000	0.5013	0.5310	0.5086	0.5051
AMEN	0.5626	0.6266	0.6154	0.7713	0.6392	0.6573	0.5577	0.5115	0.6177
Radar	0.7247	0.6587	0.6709	0.6233	0.7401	0.7399	0.5690	0.5529	0.6599
ANOMALOUS	0.7038	0.5770	0.6307	0.7316	0.7427	0.7434	0.5490	0.5535	0.6540
DOMINANT	0.7601	0.8155	0.8251	0.8081	0.7468	0.7442	0.5607	0.5163	0.7221
CoLA	0.8237	0.8779	0.8968	**0.9512**	0.7854	0.7513	0.5730	**0.5995**	0.7824
ComGA	0.8496	0.8840	0.9167	0.9220	0.8140	0.7990	0.5560	0.5587	0.7875
GRADATE	0.8580	0.9032	0.8904	0.9450	0.7519	0.7522	0.5536	0.5628	0.7771
GVE-AP	**0.9650**	**0.9216**	**0.9276**	0.9370	**0.8364**	**0.8104**	**0.5855**	0.5787	**0.8203**

4.2.1. Anomaly detection on real-world attributed networks

In this part, we test all algorithms on eight real-world networks and record their AUC results in Table 3. As shown in Table 3, GVE-AP achieves the highest AUC on six out of eight datasets. In terms of average AUC, GVE-AP also has the best rank, outperforming the second-best ComGA and third-best CoLA by 3.28 %-3.79 %. These results show the effectiveness of GVE-AP and its superiority over comparison methods. The consistent performance across datasets indicates that GVE-AP can effectively capture underlying patterns in attributed networks, which is crucial for accurate AD.

Moreover, the results also reveal that deep learning-based methods (DOMINANT, CoLA, ComGA, GRADATE, and GVE-AP) significantly outperform traditional shallow methods (LOF, AMEN, Radar, and ANOMALOUS). This trend highlights the advantages of leveraging more complex models capable of capturing the intricate structures present in real-world networks. The ability of deep models to process large-scale data and extract informative features plays a vital role in their superior performance. Notably, even on the Pubmed and Tolokers datasets, GVE-AP maintains competitive AUC performance, ranking only slightly behind CoLA or GRADATE. This performance gap can be attributed to the characteristics of these datasets, which possess abundant link structures but relatively sparse attribute information. CoLA's contrastive learning framework is particularly effective in such attribute-scarce environments. Furthermore, it is observed that when attribute information is limited (e.g., Reddit and Tolokers), the performance advantage of deep learning methods over traditional methods diminishes. This finding underscores the importance of attribute richness for anomaly detection performance, and suggests that although GVE-AP is robust, its effectiveness may vary depending on specific dataset characteristics.

Furthermore, Table 3 shows that deep learning-based methods with community structures, such as ComGA and GVE-AP, achieve significantly higher AUC values than DOMINANT when community information. This phenomenon highlights the crucial role of community structures in enhancing AD performance, as communities often exhibit distinct behavioral patterns that are critical for identifying anomalies.

Importantly, GVE-AP further outperforms ComGA. This improvement can be attributed to GVE-AP's comprehensive strategy, which jointly considers structure and attribute information during community division. By effectively modeling the interplay between node attributes and link structures, GVE-AP generates more informative community representations, enabling the design of a community-aware anomaly score function that is more powerful for detecting anomalous nodes.

Additionally, a visualization of the two detected communities on the Citeseer dataset is provided in Fig. 4. In this visualization, red nodes represent anomalous nodes, while blue and orange nodes indicate the two detected communities. It is evident that most anomalous nodes are located at the boundary between the two communities, as highlighted by the purple circle. This observation suggests that anomalies often bridge multiple communities, and community detection can effectively reveal

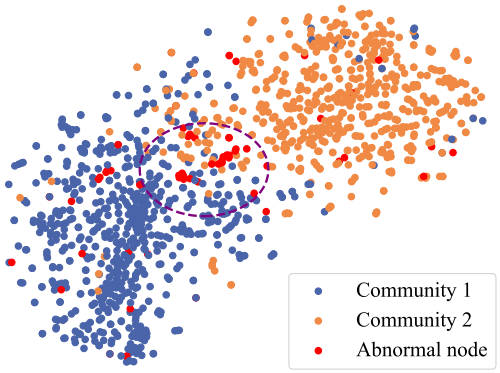


Fig. 4. Community-aware anomaly detection visualization on the citeseer dataset.

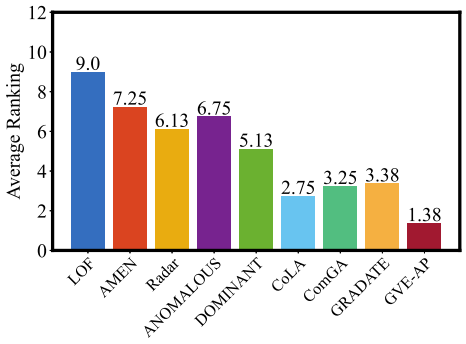


Fig. 5. Average performance ranking of nine algorithms based on AUC across eight datasets.

such structural irregularities. These results further validate the effectiveness of incorporating community structures into anomaly detection frameworks for attributed networks.

In addition, Fig. 5 presents the average rankings of the nine algorithms based on their AUC scores across the eight datasets, derived from the results in Table 3. In this case, all algorithms are assigned ranks from 1 to 10, with the best-performing algorithm receiving the lowest rank. As shown in Table 3 and Fig. 5, the proposed GVE-AP achieves the best overall rank (i.e., the lowest ranking value), outperforming recently developed methods such as GRADATE.

4.2.2. Ablation study

To verify the effectiveness of the reconstruction strategy and community-aware scoring in GVE-AP, we conduct three types of

ablation experiments on six datasets: reconstruction strategy, community anomaly scoring, and combining baseline methods with the AP algorithm.

Ablation study of reconstruction strategy. To evaluate the effectiveness of PCA and the self-attention mechanism in the reconstruction strategy, we evaluate two variants of GVE-AP: GVE-AP (w/o PCA) and GVE-AP (w/o SAM).

- **GVE-AP (w/o PCA)**: GVE-AP without the PCA dimensionality reduction mechanism.
- **GVE-AP (w/o SAM)**: GVE-AP without the self-attention mechanism.

As shown in Table 4, removing either PCA or the self-attention mechanism leads to decreased performance across all datasets. The original GVE-AP achieves the best performance on all datasets, which demonstrates that PCA effectively reduces noise and extracts meaningful features, while the self-attention mechanism captures important structural interactions, thereby improving the overall model performance.

Table 4
AUC values of GVE-AP under different reconstruction strategies.

Method	ACM	Cora	Citeseer	Pubmed	BlogCatalog	Flickr
GVE-AP (w/o PCA)	0.8516	0.8548	0.8357	0.8854	0.7823	0.7982
GVE-AP (w/o SAM)	0.9428	0.8903	0.8831	0.9120	0.8173	0.8034
GVE-AP	**0.9650**	**0.9216**	**0.9276**	**0.9370**	**0.8364**	**0.8104**

Table 5
AUC values of GVE-AP under different anomaly score settings.

Method	ACM	Cora	Citeseer	Pubmed	BlogCatalog	Flickr
GVE-AP (w/o att)	0.7193	0.7304	0.7294	0.7301	0.6330	0.5742
GVE-AP (w/o str)	0.7535	0.8369	0.8366	0.7458	0.7840	0.7445
GVE-AP	**0.9650**	**0.9216**	**0.9276**	**0.9370**	**0.8364**	**0.8104**

Ablation study of community-aware scoring. To evaluate the effectiveness of our designed anomaly score function, we evaluate two variants of GVE-AP:

- **GVE-AP (w/o att)**: GVE-AP without considering attribute anomaly scores.
- **GVE-AP (w/o str)**: GVE-AP without considering structural anomaly scores.

As shown in Table 5 and Fig. 6, removing either attribute or structural anomaly scores leads to decreased anomaly detection performance. GVE-AP considers both attribute and structural information, achieving the best performance. The results indicate that attribute and structural anomaly scores provide complementary information, enhancing the accuracy of anomaly detection.

Furthermore, on most datasets, attribute anomalies make a stronger contribution than structural anomalies, which is likely due to the fact that structural anomalies depend on global network information, making their independent detection more challenging.

Comparison with baseline methods. To further demonstrate the superiority of GVE-AP, we combine the DOMINANT model with the AP algorithm and evaluate the following variant:

- **DOMINANT + AP**: DOMINANT combined with the AP clustering algorithm. Communities are identified based on latent network embeddings extracted from the DOMINANT model, and anomaly detection is performed using our designed anomaly score function.

As shown in Table 6, compared with DOMINANT alone, combining DOMINANT with AP and the designed anomaly score function improves performance across all six datasets. This demonstrates the effectiveness of our community-aware scoring. Furthermore, GVE-AP achieves the best performance on all datasets, further validating the effectiveness and superiority of its reconstruction strategy.

4.2.3. Parameter analysis

We further investigate the influence of the hyperparameter α of GVE on the ADAN task over four real-world networks (Cora, BlogCatalog, Citeseer, and Flickr), with the results recorded in Fig. 7. As observed

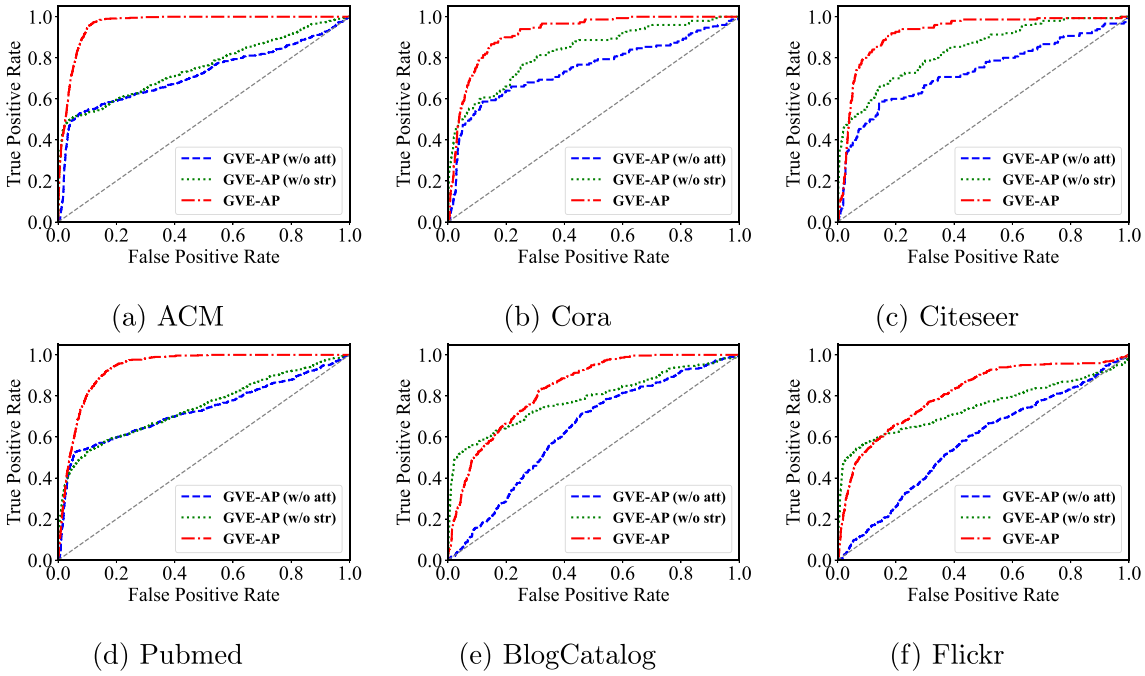


Fig. 6. ROC curves for GVE-AP, GVE-AP (w/o att), and GVE-AP (w/o str) on six real network datasets.

Table 6
Comparison of AUC values between GVE-AP and baseline methods.

Method	ACM	Cora	Citeseer	Pubmed	BlogCatalog	Flickr
DOMINANT	0.7601	0.8155	0.8251	0.8081	0.7468	0.7442
DOMINANT + AP	0.8526	0.8595	0.8827	0.8973	0.7742	0.7973
GVE-AP	**0.9650**	**0.9216**	**0.9276**	**0.9370**	**0.8364**	**0.8104**

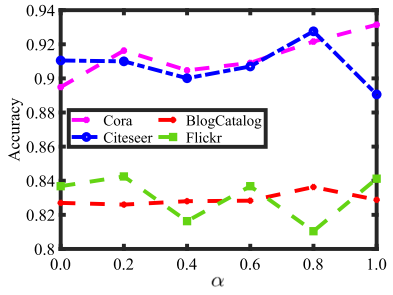


Fig. 7. The impact of the loss balancing hyperparameter α on the AUC of AD was evaluated on four datasets.

from Fig. 7, the performance of GVE-AP remains robust with respect to variations in α. More specifically, the AUC achieved by GVE-AP under different settings of α fluctuates within a margin of 4 % across all datasets. This finding demonstrates the robustness of GVE-AP in solving the ADAN task.

5. Conclusions

In this paper, we have proposed a graph variational autoencoder with affinity propagation (GVE-AP) for community-aware AD in attributed networks, which considers the effects of community structures on the ADAN task. GVE-AP first uses a GVE module to learn essential node representations by jointly considering link structures and node attributes. Then, it employs an AP algorithm to detect communities within the attributed networks. Subsequently, it proposes a novel community-aware anomaly score to identify anomalous nodes by capturing their abnormal behaviors within communities. Experiments on eight real-world attributed networks demonstrate that the proposed GVE-AP outperforms eight state-of-the-art methods in terms of AUC. The results also validate the effectiveness of the designed anomaly score and the robustness of GVE-AP for the ADAN task. In the future, we will study community-aware anomaly detection in various types of network data, such as dynamic networks and hypergraphs.

CRediT authorship contribution statement

Zhijie Cao: Writing – review & editing, Writing – original draft, Project administration. **Chengkun Yang:** Validation, Data curation. **Xiaoqing Fan:** Methodology, Conceptualization. **Lingjie Li:** Project administration. **Qiuzhen Lin:** Project administration. **Jianqiang Li:** Project administration. **Lijia Ma:** Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This work was supported by the National Natural Science Foundation of China under Grant 62173236, the Shenzhen Natural Science Foundation under Grant JCYJ20240813141416022, and the Tencent “Rhinoceros Bird” Scientific Research Foundation for Young Teachers of ShenZhen University.

Data availability

Data will be made available on request.

References

[1] Z. Wang, D. Zhang, X. Zhou, D. Yang, Z. Yu, Z. Yu, Discovering and profiling overlapping communities in location-based social networks, IEEE Trans. Syst. Man Cybern. Syst. 44 (4) (2014) 499–509.
[2] A.-L. Barabasi, Z.N. Oltvai, Network biology: understanding the cell’s functional organization, Nat. Rev. Genet. 5 (2) (2004) 101–113.
[3] Q. Zhong, Y. Liu, X. Ao, B. Hu, J. Feng, J. Tang, Q. He, Financial defaulter detection on online credit payment VIA multi-view attributed heterogeneous information network, in: Proceedings of the Web Conference 2020, ACM, Taipei, Taiwan, 2020, pp. 785–795.
[4] F.E. Grubbs, Procedures for detecting outlying observations in samples, Technometrics 11 (1) (1969) 1–21.
[5] X. Ma, J. Wu, S. Xue, J. Yang, C. Zhou, Q.Z. Sheng, H. Xiong, L. Akoglu, A comprehensive survey on graph anomaly detection with deep learning, IEEE Trans. Knowl. Data Eng. 35 (12) (2023) 12012–12038.
[6] B. Branco, P. Abreu, A.S. Gomes, M.S.C. Almeida, J.T. Ascensão, P. Bizarro, Interleaved sequence RNNS for fraud detection, in: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, ACM, Virtual Event, CA, USA, 2020, pp. 3101–3109.
[7] S. Ahmed, K. Hinkelmann, F. Corradini, Combining machine learning with knowledge engineering to detect fake news in social networks-a survey, arXiv preprint arXiv:2201.08032, 2022.
[8] N. Shone, T.N. Ngoc, V.D. Phai, Q. Shi, A deep learning approach to network intrusion detection, IEEE Trans. Emerg. Top. Comput. Intell. 2 (1) (2018) 41–50.
[9] M.M. Breunig, H.-P. Kriegel, R.T. Ng, J. Sander, LOF: identifying density-based local outliers, in: Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data, ACM, Dallas, Texas, USA, 2000, pp. 93–104.
[10] J. Gao, F. Liang, W. Fan, C. Wang, Y. Sun, J. Han, On community outliers and their efficient detection in information networks, in: Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, ACM, Washington, DC, USA, 2010, pp. 813–822.
[11] J. Li, H. Dani, X. Hu, H. Liu, Radar: residual analysis for anomaly detection in attributed networks, in: Proceedings of the 26th International Joint Conference on Artificial Intelligence, AAAI Press, Melbourne, Australia, 2017, pp. 2152–2158.
[12] L. Akoglu, M. McGlohon, C. Faloutsos, Oddball: spotting anomalies in weighted graphs, in: Proceeding of the Advances in Knowledge Discovery and Data Mining, 14th Pacific-Asia Conference, Springer, Hyderabad, India, 2010, pp. 410–421.
[13] K. Ding, J. Li, R. Bhanushali, H. Liu, Deep anomaly detection on attributed networks, in: Proceedings of the 2019 SIAM International Conference on Data Mining, SIAM, Calgary, Alberta, Canada, 2019, pp. 594–602.
[14] Y. Li, X. Huang, J. Li, M. Du, N. Zou, Specae: spectral autoencoder for anomaly detection in attributed networks, in: Proceedings of the 28th ACM International Conference on Information and Knowledge Management, ACM, Beijing, China, 2019, pp. 2233–2236.
[15] P. Veličković, W. Fedus, W.L. Hamilton, P. Liò, Y. Bengio, R.D. Hjelm, Deep graph infomax, arXiv preprint arXiv:1809.10341, 2018.
[16] M. Munir, S.A. Siddiqui, A. Dengel, S. Ahmed, Deepant: a deep learning approach for unsupervised anomaly detection in time series, IEEE Access 7 (2019) 1991–2005.
[17] H. Fan, F. Zhang, Z. Li, Anomalydae: dual autoencoder for anomaly detection on attributed networks, in: ICASSP 2020 - 2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), IEEE, Barcelona, Spain, 2020, pp. 5685–5689.
[18] Y. Liu, Z. Li, S. Pan, C. Gong, C. Zhou, G. Karypis, Anomaly detection on attributed networks VIA contrastive self-supervised learning, IEEE Trans. Neural Netw. Learn. Syst. 33 (6) (2021) 2378–2392.
[19] X. Luo, J. Wu, A. Beheshti, J. Yang, X. Zhang, Y. Wang, S. Xue, Comga: community-aware attributed graph anomaly detection, in: Proceedings of the 15th ACM International Conference on Web Search and Data Mining, ACM, Tempe, AZ, USA, 2022, pp. 657–665.
[20] Z. Peng, M. Luo, J. Li, L. Xue, Q. Zheng, A deep multi-view framework for anomaly detection on attributed networks, IEEE Trans. Knowl. Data Eng. 34 (6) (2022) 2539–2552.
[21] C. Xiao, X. Xu, Y. Lei, K. Zhang, S. Liu, F. Zhou, Counterfactual graph learning for anomaly detection on attributed networks, IEEE Trans. Knowl. Data Eng. 35 (10) (2023) 10540–10553.
[22] Q. Wang, G. Pang, M. Salehi, W. Buntine, C. Leckie, Cross-domain graph anomaly detection VIA anomaly-aware contrastive alignment, in: Proceedings of the AAAI Conference on Artificial Intelligence, AAAI Press, Washington, DC, USA, 2023, pp. 4676–4684.
[23] K. Suresh, K. Jayasakthi Velmurugan, R. Vidhya, S. Rahini Sudha, Kavitha, Deep anomaly detection: a linear one-class SVM approach for high-dimensional and large-scale data, Appl. Soft Comput. (2024) 112369.
[24] S. Pan, R. Hu, G. Long, J. Jiang, L. Yao, C. Zhang, Adversarially regularized graph autoencoder for graph embedding, arXiv preprint arXiv:1802.04407, 2018.
[25] C. Gao, J. Zhu, F. Zhang, Z. Wang, X. Li, A novel representation learning for dynamic graphs based on graph convolutional networks, IEEE Trans. Cybern. 53 (6) (2023) 3599–3612.
[26] H. Wang, J. Wu, W. Hu, X. Wu, Detecting and assessing anomalous evolutionary behaviors of nodes in evolving social networks, ACM Trans. Knowl. Discov. Data 13 (1) (2019) 1–24.

[27] M. Girvan, M.E.J. Newman, Community structure in social and biological networks, Proc. Natl. Acad. Sci. 99 (12) (2002) 7821–7826.
[28] J. Yang, J. McAuley, J. Leskovec, Community detection in networks with node attributes, in: Proceeding of the 2013 IEEE 13th International Conference on Data Mining, IEEE, Dallas, TX, USA, 2013, pp. 1151–1156.
[29] Z. Li, J. Liu, K. Wu, A multiobjective evolutionary algorithm based on structural and attribute similarities for community detection in attributed networks, IEEE Trans. Cybern. 48 (7) (2018) 1963–1976.
[30] J. Zhu, C. Wang, C. Gao, F. Zhang, Z. Wang, X. Li, Community detection in graph: an embedding method, IEEE Trans. Netw. Sci. Eng. 9 (2) (2022) 689–702.
[31] R. Shang, W. Zhang, L. Jiao, X. Zhang, R. Stolkin, Dynamic immunization node model for complex networks based on community structure and threshold, IEEE Trans. Cybern. 52 (3) (2022) 1539–1552.
[32] R. Shang, S. Wang, W. Zhang, J. Feng, L. Jiao, R. Stolkin, Evolutionary multi-objective overlapping community detection based on fusion of internal and external connectivity and correction of node intimacy, Appl. Soft Comput. 154 (2024) 111414.
[33] X. Wang, D. Jin, X. Cao, L. Yang, W. Zhang, Semantic community identification in large attribute networks, in: Proceedings of the AAAI Conference on Artificial Intelligence, AAAI Press, Phoenix, Arizona, USA, 2016, pp. 265–271.
[34] F. Wang, T. Li, X. Wang, S. Zhu, C. Ding, Community discovery using nonnegative matrix factorization, Data Min. Knowl. Discov. 22 (3) (2011) 493–521.
[35] R. Shang, W. Zhang, J. Zhang, L. Jiao, Y. Li, R. Stolkin, Local community detection algorithm based on alternating strategy of strong fusion and weak fusion, IEEE Trans. Cybern. 53 (2) (2023) 818–831.
[36] C. Gao, Z. Yin, Z. Wang, X. Li, X. Li, Multilayer network community detection: a novel multi-objective evolutionary algorithm based on consensus prior information, IEEE Comput. Intell. Mag. 18 (2) (2023) 46–59.
[37] Y. Li, C. Sha, X. Huang, Y. Zhang, Community detection in attributed graphs: an embedding approach, in: Proceedings of the 32nd AAAI Conference on Artificial Intelligence, AAAI Press, New Orleans, Louisiana, USA, 2018, pp. 338–345.
[38] H. Sun, F. He, J. Huang, Y. Sun, Y. Li, C. Wang, L. He, Z. Sun, X. Jia, Network embedding for community detection in attributed networks, ACM Trans. Knowl. Discov. Data 14 (3) (2020) 1–25.
[39] Z. Huang, X. Zhong, Q. Wang, M. Gong, X. Ma, Detecting community in attributed networks by dynamically exploring node attributes and topological structure, Knowl.-Based Syst. 196 (2020) 105760.
[40] R. Francisquini, A.C. Lorena, M.C.V. Nascimento, Community-based anomaly detection using spectral graph filtering, Appl. Soft Comput. 118 (2022) 108489.
[41] C. Liu, Y. Yang, Y. Ding, H. Lu, W. Lin, Z. Wu, W. Bi, DAG: deep adaptive and generative k-free community detection on attributed graphs, in: Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, ACM, Barcelona, Spain, 2024, pp. 5454–5465.
[42] S. Sobolevsky, A. Belyi, Graph neural network inspired algorithm for unsupervised network community detection, Appl. Netw. Sci. 7 (1) (2022) 63.
[43] T.N. Kipf, M. Welling, Variational graph auto-encoders, arXiv preprint arXiv:1611.07308, 2016.
[44] S. Wold, K. Esbensen, P. Geladi, Principal component analysis, Chemom. Intell. Lab. Syst. 2 (1) (1987) 37–52.
[45] B.J. Frey, D. Dueck, Clustering by passing messages between data points, Science 315 (5814) (2007) 972–976.
[46] N. Li, L.J. Latecki, Affinity learning for mixed data clustering, in: Proceedings of the 26th International Joint Conference on Artificial Intelligence, Ijcai.org, Melbourne, Australia, 2017, pp. 2173–2179.
[47] B. Perozzi, L. Akoglu, Scalable anomaly ranking of attributed neighborhoods, in: Proceedings of the 2016 SIAM International Conference on Data Mining, SIAM, Miami, Florida, USA, 2016, pp. 207–215.
[48] Z. Peng, M. Luo, J. Li, H. Liu, Q. Zheng, et al., Anomalous: a joint modeling approach for anomaly detection on attributed networks, in: Proceedings of the 27th International Joint Conference on Artificial Intelligence, vol. 18, Ijcai.org, Stockholm, Sweden, 2018, pp. 3513–3519.
[49] J. Duan, S. Wang, P. Zhang, E. Zhu, J. Hu, H. Jin, Y. Liu, Z. Dong, Graph anomaly detection VIA multi-scale contrastive learning networks with augmented view, in: Proceeding of the 37th AAAI Conference on Artificial Intelligence, AAAI Press, Washington, DC, USA, 2023, pp. 7459–7467.
[50] J. Pan, Y. Liu, Y. Zheng, S. Pan, Prem: a simple yet effective approach for node-level graph anomaly detection, in: Proceeding of the 2023 IEEE International Conference on Data Mining (ICDM), IEEE, Shanghai, China, 2023, pp. 1253–1258.
[51] X. Wang, S. Lai, S. Zhu, Y. Chen, L. Lv, Y. Qi, Graph anomaly detection VIA multiscale contrastive self-supervised learning from local to global, IEEE Trans. Comput. Soc. Syst. 12 (2) (2025) 485–497.
[52] S. Liu, D. Yao, L. Fang, Z. Li, W. Li, K. Feng, X. Ji, J. Bi, Anomalyllm: few-shot anomaly edge detection for dynamic graphs using large language models, arXiv preprint arXiv:2405.07626, 2024.
[53] X. Han, S. Yuan, M. Trabelsi, Loggpt: log anomaly detection VIA GPT, in: Proceeding of the 2023 IEEE International Conference on Big Data (BigData), IEEE, Sorrento, Italy, 2023, pp. 1117–1122.
[54] D. Jin, Z. Yu, P. Jiao, S. Pan, D. He, J. Wu, P.S. Yu, W. Zhang, A survey of community detection approaches: from statistical modeling to deep learning, IEEE Trans. Knowl. Data Eng. 35 (2) (2023) 1149–1170.
[55] K. Ding, J. Li, N. Agarwal, H. Liu, Inductive anomaly detection on attributed networks, in: Proceedings of the 29th International Joint Conference on Artificial Intelligence, Ijcai.org, Yokohama, Japan, 2021, pp. 1288–1294.
[56] J. Tang, F. Hua, Z. Gao, P. Zhao, J. Li, Gadbench: revisiting and benchmarking supervised graph anomaly detection, in: Proceeding of the Advances in Neural Information Processing Systems, Curran Associates, Inc., New Orleans, LA, USA, 2023, pp. 29628–29653.